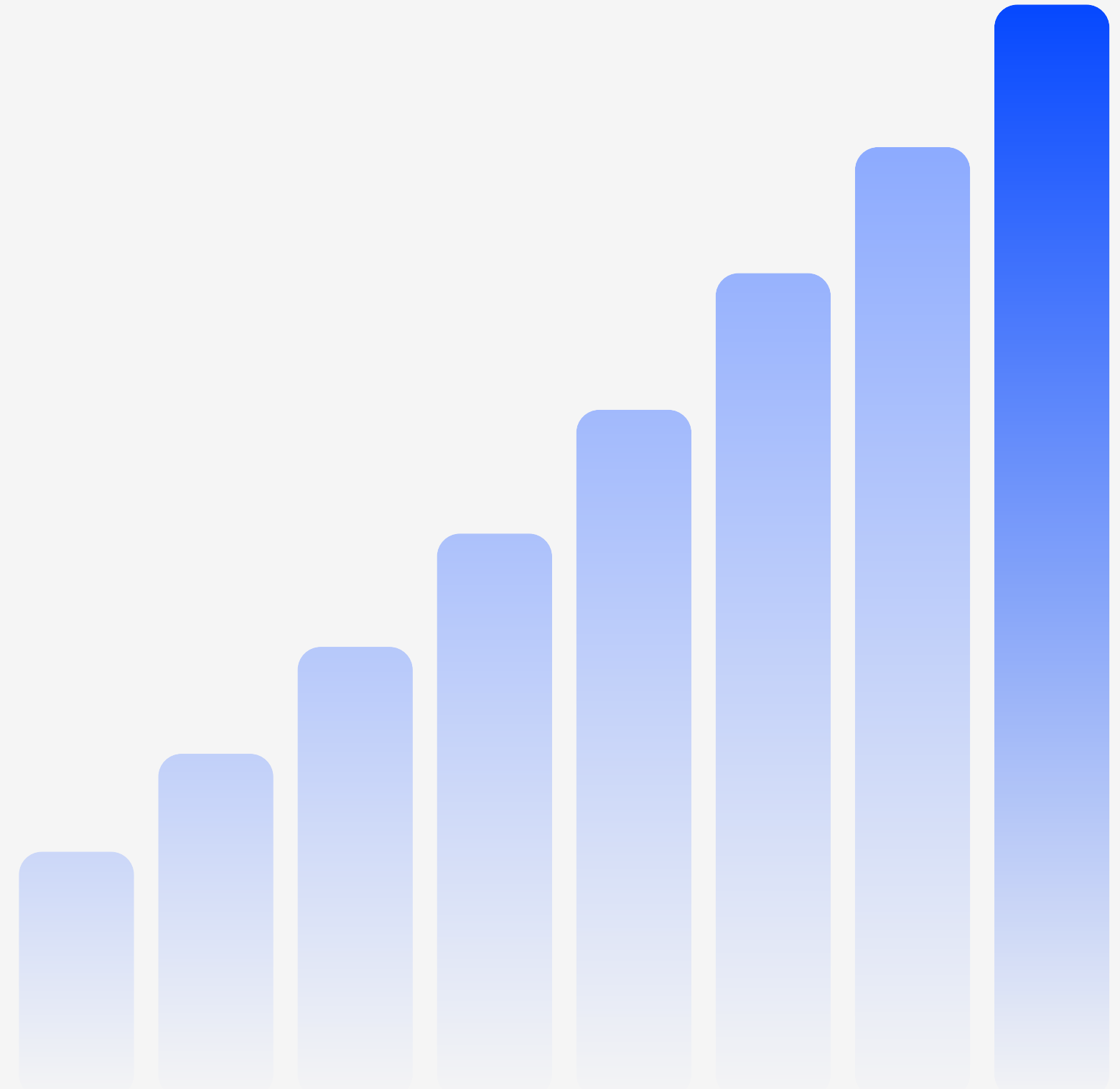


공공데이터 융합 풀스택 개발자 양성과정

서버프로그램 구현



목 차

01	ProductDTO 정승원	05	ProductRestController 강현욱
02	ProductRepository 정승원	06	ProductBatch 박형규
03	ProductServiceImpl 김정희	07	ProductBatchTest 박형규
04	ProductServiceTest 김정희	08	화면구현

```
// TODO: DTO → Entity 변환 메서드를 구현하라.  
// - Product.builder() 로 id/name/price/stock/category/createAt 를 매핑  
public Product toEntity() { 0개의 사용위치 jsh340866 +1  
    throw new UnsupportedOperationException("TODO: toEntity 구현");  
    return Product.builder()  
        .id(this.id)  
        .name(this.name)  
        .price(this.price)  
        .stock(this.stock)  
        .category(this.category)  
        .createAt(this.createAt)  
        .build();  
}
```

toEntity() : DTO → Entity 변환

- 현재 ProductDTO 객체의 데이터를 Product 엔티티 객체로 변환
- Product.builder()를 이용하여 id, name, price, stock, category, createAt 값을 매핑
- DTO에 저장된 값을 Entity에 복사하여 새로운 Product 객체 생성
- JpaRepository의 save() 메서드에서 사용할 수 있는 형태로 변환

```
// TODO: Entity → DTO 변환(static) 메서드를 구현하라.  
// - ProductDTO.builder() 로 Product 의 각 필드를 매핑하여 ProductDTO  
public static ProductDTO from(Product p) { 0개의 사용위치 jsh340866  
    throw new UnsupportedOperationException("TODO: from 구현");  
    return ProductDTO.builder()  
        .id(p.getId())  
        .name(p.getName())  
        .price(p.getPrice())  
        .stock(p.getStock())  
        .category(p.getCategory())  
        .createAt(p.getCreateAt())  
        .build();  
}
```

from(Product p) : Entity → DTO 변환

- Product 엔티티 객체를 ProductDTO 객체로 변환
- ProductDTO.builder()를 이용하여 Entity의 각 필드를 DTO에 매핑
- DB 조회 결과를 화면 또는 API 응답용 DTO로 변환
- static 메서드로 구현되어 DTO 객체 생성 없이 호출 가능

02 | 코드구현 - ProductRepository 정승원

```
@Repository 6개 사용 위치 Ⓜjsh340866 +1
public interface ProductRepository extends JpaRepository<Product, Long> {

    // TODO: 등록 시 상품명 중복 검사용 쿼리메서드를 선언하라.
    //      (힌트: boolean existsByName(String name) - 선언만 하면 구현은 자동 생성)
    boolean existsByName(String name); 0개의 사용위치 Ⓜjsh340866

    // TODO: 수정 시 자기 자신(id)을 제외한 상품명 중복 검사용 쿼리메서드를 선언하라.
    //      (힌트: boolean existsByNameAndIdNot(String name, Long id))
    boolean existsByNameAndIdNot(String name, Long id); 0개의 사용위치 Ⓜjsh340866

    // TODO(배치): 품절(stock=0) 상품 건수 조회용 쿼리메서드를 선언하라.
    //      (힌트: long countByStock(int stock))
    long countByStock(int stock); 0개의 사용위치 Ⓜjsh340866
}
```

ProductRepository

- JpaRepository<Product, Long> 상속
- Product 엔티티에 대한 CRUD 기능 제공
- PK 타입으로 Long 사용
- 메서드 이름만 선언하면 SQL 자동 생성
- 별도의 구현 클래스 작성 불필요
- Repository 계층에서 DB 접근 담당

```
// TODO: 등록 시 상품명 중복 검사용 쿼리메서드를 선언하라.  
//      (힌트: boolean existsByName(String name) - 선언만 하면  
boolean existsByName(String name); 0개의 사용위치 👤jsh340866
```

existsByName(String name)

- 상품 등록 시 상품명 중복 여부 확인
- 전달받은 상품명(name)과 동일한 데이터가 존재하는지 조회
- 존재하면 true, 존재하지 않으면 false 반환

```
// TODO: 수정 시 자기 자신(id)을 제외한 상품명 중복 검사용 쿼리메서드를 선언  
//      (힌트: boolean existsByNameAndIdNot(String name, Long id))  
boolean existsByNameAndIdNot(String name, Long id); 0개의 사용위치
```

existsByNameAndIdNot(String name, Long id)

- 상품 수정 시 상품명 중복 여부 확인
- 현재 수정 중인 상품의 id는 제외하고 조회
- 다른 상품이 동일한 상품명을 사용하는지 검사
- 자기 자신을 중복으로 판단하는 문제 방지

```
// TODO(배치): 품절(stock=0) 상품 건수 조회용 쿼리메서드를 선언  
//      (힌트: long countByStock(int stock))  
long countByStock(int stock); 0개의 사용위치 ㄹjsh340866
```

countByStock(int stock)

- 특정 재고 수량(stock)을 가진 상품 개수 조회
- 주로 배치(Batch) 작업에서 품절 상품 수 집계에 사용
- 재고가 0인 상품 개수 조회 가능
- 조회 결과를 long 타입으로 반환

03 | 코드구현 - ProductServiceImpl 김정희

```
@Override
@Transactional
public ProductDTO register(ProductDTO dto) {
    if(productRepository.existsByName(dto.getName())) throw new MyBizException("중복된 상품 이름입니다.");
    dto.setCreateAt(LocalDateTime.now());
    return ProductDTO.from(productRepository.save(dto.toEntity()));
}
```

```
@Override
@Transactional
public ProductDTO modify(Long id, ProductDTO dto) {
    Optional<Product> p = productRepository.findById(id);
    if(p.isEmpty()) throw new MyBizException("존재하지 않는 상품id입니다.");
    if(productRepository.existsByNameAndIdNot(dto.getName(),id)) throw new MyBizException("동일한 상품이 존재합니다.");
    p.get().setName(dto.getName());
    p.get().setCategory(dto.getCategory());
    p.get().setStock(dto.getStock());
    p.get().setPrice(dto.getPrice());
    p.get().setCreateAt(LocalDateTime.now());

    return ProductDTO.from(p.get());
}
```

상품 등록

@Transactional로 트랜잭션 적용해주고
existsByName으로 동일 이름이 존재한다면 MyBizException으로 throw
없다면 현재시간 반영 후 Entity타입으로 변환 후 저장하여 반환 받는 값을 DTO타입으로 변환 후 반환

상품 수정

@Transactional로 트랜잭션 적용해주고
findById로 찾아서 Optional<Product> 타입으로 저장한 후 isEmpty를 사용하여 값이 비어있다면 MyBizException으로 throw
있다면 dirty checking으로 업데이트 후 값을 DTO타입으로 반환

03 | 코드구현 - ProductServiceImpl 김정희

```
@Override
@Transactional
public void remove(Long id) {
    if(!productRepository.existsById(id)) throw new MyBizException("존재하지 않는 상품id입니다.");
    productRepository.deleteById(id);
}
```

```
@Override
@Transactional(readonly = true)
public ProductDTO get(Long id) {
    Optional<Product> p = productRepository.findById(id);
    if(p.isEmpty()) throw new MyBizException("존재하지 않는 상품id입니다.");
    return ProductDTO.from(p.get());
}
```

```
@Override
@Transactional(readonly = true)
public Page<ProductDTO> list(Pageable pageable) {
    return productRepository.findAll(pageable).map(ProductDTO::from);
}
```

상품 삭제

@Transactional로 트랜잭션 적용해주고
existsById로 상품이 존재하지 않으면 MyBizException으로
throw
있다면 deleteById를 통해 해당 id의 데이터 삭제

상품 단건 조회

@Transactional로 트랜잭션 적용해주고 readOnly 를 true로 설정
findById로 찾아서 Optional<Product> 타입으로 저장한 후
isEmpty를 사용하여 값이 비어있다면 MyBizException으로
throw
있다면 값을 DTO타입으로 반환

상품 목록 조회

@Transactional로 트랜잭션 적용해주고 readOnly 를 true로 설정
productRepository.findAll(pageable) 결과를
.map(ProductDTO::from) 으로 변환하여 반환

03 | 코드구현 - ProductServiceImpl 김정희

```
@Override
@Transactional
public int registerBulk(List<ProductDTO> list) {
    int count=0;
    for(ProductDTO dto : list) {
        if (productRepository.existsByName(dto.getName())) throw new MyBizException("동일한 상품명에 존재합니다.");
        dto.setCreatedAt(LocalDateTime.now());
        productRepository.save(dto.toEntity());
        count++;
    }
    return count;
}
```

일괄 등록

@Transactional을 적용하여 하나의 트랜잭션으로 처리하고, 중간에 오류 발생 시 전체 롤백되도록 구현

저장된 상품 개수를 저장하기 위한 count 변수를 0으로 초기화
전달받은 List<ProductDTO>를 for문으로 순회하며 상품을 순차적으로 등록

existsByName()을 통해 동일한 상품명인지 확인하고, 중복 시 MyBizException을 발생

중복이 없는 경우 현재 시간을 createdAt에 저장한 뒤 Entity로 변환하여 DB에 저장하고 count를 1 증가

모든 상품이 정상적으로 저장되면 등록된 상품 개수(count)를 반환

04 | 코드구현 - ProductServiceTest 김정희

```
@Test
void register_정상() {
    ProductDTO dto = productService.register(ProductDTO.builder()
        .name("A")
        .price(1000)
        .stock(10)
        .category("AA")
        .build());
    assertNotNull(dto.getId());
}

@Test
void registerBulk_중간실패_전체롤백() {
    productService.register(ProductDTO.builder()
        .name("A")
        .price(1000)
        .stock(10)
        .category("AA")
        .build());
    List<ProductDTO> list = new ArrayList<>();
    list.add(ProductDTO.builder()
        .name("B")
        .price(1000)
        .stock(10)
        .category("BB")
        .build());
    list.add(ProductDTO.builder()
        .name("A")
        .price(1000)
        .stock(10)
        .category("AA")
        .build());
    assertThrows(MyBizException.class, () -> {System.out.println("등록 수" + productService.registerBulk(list));});
    assertEquals( expected: 3, productRepository.count());
}
```

상품 등록 테스트

builder를 통해 값을 입력 후 저장

일괄 등록 테스트

builder를 통해 값을 미리 입력 후 저장

리스트 생성 후 리스트에 값을 입력하며 두번째 입력시 미리 입력해둔 값의 이름을 동일하게 리스트에 저장

리스트를 일괄 등록 후 MyBizException이 안뜨면 count수를 출력

MyBizException 예외 발생 시 assertEquals를 통해 DB의 저장된 값이 3인지 확인

DB에 테이블 내의 데이터가 없다고 가정할 때

트랜잭션이 적용되었다면 실제값 1

실패시 실제값 2

05 | 코드구현 - ProductRestController 강현욱

```
@Autowired
private ProductService productService;

// TODO: 목록 조회
// - productService.list(pageable) 호출
// - 결과의 content / number / totalPages / totalElements 를 Map 에 담아
//   ResponseEntity 로 200(OK) 반환
@GetMapping(produces = MediaType.APPLICATION_JSON_VALUE) @kanghyunuk
public ResponseEntity<Map<String, Object>> list(Pageable pageable) {

    Page<ProductDTO> page = productService.list(pageable);

    Map<String, Object> responseMap = new HashMap<>();

    responseMap.put("content", page.getContent());
    responseMap.put("number", page.getNumber());
    responseMap.put("totalPages", page.getTotalPages());
    responseMap.put("totalElements", page.getTotalElements());

    return ResponseEntity.status(HttpStatus.OK).body(responseMap);
}
```

목록 조회

- 상품 목록을 페이지 단위로 조회하고 페이징 정보를 함께 반환한다.
- Pageable 객체를 이용하여 페이지 번호와 페이지 크기 정보를 전달받는다.
- Controller에서 productService.list(pageable)를 호출하여 상품 목록 조회를 요청한다.
- Service는 조회 결과를 Page<ProductDTO> 형태로 반환한다.
- 조회된 목록(content)과 페이징 정보(number, totalPages, totalElements)를 Map에 저장한다.
- ResponseEntity를 이용하여 200(OK) 상태 코드와 함께 JSON 데이터를 반환한다.

05 | 코드구현 - ProductRestController 강현욱

```
// TODO: 단건 조회
// - productService.get(id) 결과를 200(OK) 으로 반환
@GetMapping(value =("/{id}", produces = MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<ProductDTO> get(@PathVariable("id") Long id) {

    ProductDTO dto = productService.get(id);

    return ResponseEntity.status(HttpStatus.OK).body(dto);
}
```

단건 조회

- 상품 ID를 이용하여 특정 상품의 상세 정보를 조회한다.
- @PathVariable을 사용하여 URL 경로의 id 값을 전달받는다.
- Controller에서 productService.get(id)를 호출하여 상품 조회를 요청한다.
- Service는 조회된 상품 정보를 ProductDTO 형태로 반환한다.
- 반환받은 ProductDTO를 ResponseEntity에 담아 응답한다.
- 조회 성공 시 200(OK) 상태 코드와 함께 상품 정보를 JSON 형태로 반환한다.

05 | 코드구현 - ProductRestController 강현욱

```
// TODO: 등록
// - @RequestBody @Valid ProductDTO + BindingResult 로 검증
// - bindingResult.hasErrors() 이면 FieldError 들을 들어 필드별 메시지를 Map 에 담아 400(BAD_REQUEST) 반환
// - 통과 시 productService.register(dto) 호출 후 200 + "상품 등록 성공!" 메시지 반환
@PostMapping(consumes = MediaType.APPLICATION_JSON_VALUE) @ResponseBody
public ResponseEntity<Map<String, Object>> add(@RequestBody @Valid ProductDTO dto,
                                              BindingResult bindingResult) {

    Map<String, Object> responseMap = new HashMap<>();

    // 유효성 검증
    if(bindingResult.hasErrors()){
        for(FieldError error : bindingResult.getFieldErrors()){
            responseMap.put(
                error.getField(),
                error.getDefaultMessage()
            );
        }

        return ResponseEntity
            .status(HttpStatus.BAD_REQUEST)
            .body(responseMap);
    }

    // 서비스 실행
    productService.register(dto);

    // 성공 응답
    responseMap.put("message", "상품 등록 성공!");

    return ResponseEntity
        .status(HttpStatus.OK)
        .body(responseMap);
}
```

상품 등록

- 클라이언트가 전달한 상품 정보를 @RequestBody를 통해 ProductDTO로 전달받는다.
- @Valid와 BindingResult를 이용하여 입력값에 대한 유효성 검증을 수행한다.
- 검증 오류가 발생하면 FieldError를 순회하며 필드별 오류 메시지를 Map에 저장한다.
- 유효성 검증 실패 시 400(BAD_REQUEST) 상태 코드와 오류 메시지를 반환한다.
- 검증을 통과하면 productService.register(dto)를 호출하여 상품을 등록한다.
- 등록 완료 후 "상품 등록 성공!" 메시지를 Map에 저장한다.
- ResponseEntity를 이용하여 200(OK) 상태 코드와 함께 성공 메시지를 반환한다.

05 | 코드구현 - ProductRestController 강현욱

```
// TODO: 수정
// - @PathVariable id + @RequestBody @Valid ProductDTO + BindingResult
// - 검증 오류 처리는 등록과 동일 / 통과 시 productService.modify(id, dto) → 200 + "상품 수정 성공!"
@PutMapping(value = "/{id}", consumes = MediaType.APPLICATION_JSON_VALUE) ㄹ kanghyunuk
public ResponseEntity<Map<String, Object>> update(@PathVariable("id") Long id,
                                                  @RequestBody @Valid ProductDTO dto,
                                                  BindingResult bindingResult) {

    Map<String, Object> responseMap = new HashMap<>();

    // 유효성 검증
    if (bindingResult.hasErrors()) {
        for (FieldError error : bindingResult.getFieldErrors()) {
            responseMap.put(error.getField(), error.getDefaultMessage());
        }

        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(responseMap);
    }

    // 수정
    productService.modify(id, dto);

    // 성공 응답
    responseMap.put("message", "상품 수정 성공!");

    return ResponseEntity.status(HttpStatus.OK).body(responseMap);
}
```

상품 수정

- @PathVariable을 이용하여 수정할 상품의 ID를 전달받는다.
- @RequestBody를 통해 수정할 상품 정보를 ProductDTO로 전달받는다.
- @Valid와 BindingResult를 이용하여 입력값에 대한 유효성 검증을 수행한다.
- 검증 오류가 발생하면 FieldError를 순회하며 필드별 오류 메시지를 Map에 저장한다.
- 유효성 검증 실패 시 400(BAD_REQUEST) 상태 코드와 오류 메시지를 반환한다.
- 검증을 통과하면 productService.modify(id, dto)를 호출하여 상품 정보를 수정한다.
- 수정 완료 후 "상품 수정 성공!" 메시지를 Map에 저장한다.
- ResponseEntity를 이용하여 200(OK) 상태 코드와 함께 성공 메시지를 반환한다.

05 | 코드구현 - ProductRestController 강현욱

```
// TODO: 삭제
// - productService.remove(id) 호출 후 200 + "상품 삭제 성공!" 메시지 반환
@DeleteMapping("/{id}") @Log(kanghyunuk)
public ResponseEntity<Map<String, Object>> delete(@PathVariable("id") Long id) {
    Map<String, Object> responseMap = new HashMap<>();

    productService.remove(id);

    responseMap.put("message", "상품 삭제 성공!");

    return ResponseEntity.status(HttpStatus.OK).body(responseMap);
}
```

상품 삭제

- @PathVariable을 이용하여 삭제할 상품의 ID를 전달받는다.
- Controller에서 productService.remove(id)를 호출하여 상품 삭제를 요청한다.
- Service 계층에서 해당 상품 데이터를 삭제한다.
- 삭제 완료 후 "상품 삭제 성공!" 메시지를 Map에 저장한다.
- ResponseEntity를 이용하여 200(OK) 상태 코드와 함께 성공 메시지를 반환한다.

05 | 코드구현 - ProductRestController 강현욱

```
// TODO: 일괄 등록 (한 트랜잭션, 중간 실패 시 전체 롤백)
// - @RequestBody List<ProductDTO>
// - productService.registerBulk(list) 호출 후 등록 건수(count) + "일괄 등록 성공!" 반환
@PostMapping(value = "/bulk", consumes = MediaType.APPLICATION_JSON_VALUE) @kanghyunuk
public ResponseEntity<Map<String, Object>> addBulk(@RequestBody List<ProductDTO> list) {
    Map<String, Object> responseMap = new HashMap<>();

    int count = productService.registerBulk(list);

    responseMap.put("count", count);
    responseMap.put("message", "일괄 등록 성공!");

    return ResponseEntity.status(HttpStatus.OK).body(responseMap);
}
```

상품 일괄 등록

- @RequestBody를 이용하여 여러 개의 상품 정보를 List<ProductDTO> 형태로 전달받는다.
- Controller에서 productService.registerBulk(list)를 호출하여 상품 일괄 등록을 요청한다.
- Service 계층에서 여러 상품을 하나의 트랜잭션으로 처리한다.
- 등록이 완료되면 등록된 상품 개수(count)를 반환받는다.
- 등록 건수와 "일괄 등록 성공!" 메시지를 Map에 저장한다.
- ResponseEntity를 이용하여 200(OK) 상태 코드와 함께 결과를 반환한다.

06 | 코드구현 - ProductBatch 박형규

```
@Scheduled(fixedDelayString = "${batch.soldout.delay:60000}") @kanghyunuk
@Transactional(readOnly = true)
public void scheduleSoldOutCheck() {
    int count = checkSoldOut();
    log.info("[Batch] 품절 점검 완료 - 품절 상품 수=" + count);
}

/*
 * TODO: ★학생 작업★ 품절(stock = 0) 상품의 "건수"를 조회하여 int 로 반환하라.
 *
 * [해야 할 일 - 단계별]
 * 1) 이 메서드 위에 @Transactional(readOnly = true) 애너테이션을 붙인다.
 *    → 조회 전용 트랜잭션 안에서 DB 에 안전하게 접근하기 위함.
 * 2) ProductRepository 에 품절 건수 조회용 쿼리메서드를 선언한다.
 *    → 예: long countByStock(int stock) (Repository 파일의 TODO 참고, 선언만 하면 자동 구현)
 * 3) 이 본문에서 productRepository.countByStock(0) 을 호출하고,
 *    그 결과(long)를 int 로 변환하여 return 한다.
 *
 * [동작 흐름]
 * scheduleSoldOutCheck() (자동) → checkSoldOut() (여기) → countByStock(0) → 품절 건수 반환 → 로그 출력
 */
public int checkSoldOut() { 2개 사용 위치 @kanghyunuk +1
    return (int)productRepository.countByStock(0);
}
```

```
Repository 6개 사용 위치 @kanghyunuk +1
public interface ProductRepository extends JpaRepository<Product, Long> {

    // TODO: 등록 시 상품명 중복 검사용 쿼리메서드를 선언하라.
    // (힌트: boolean existsByName(String name) - 선언만 하면 구현은 자동 생성)
    boolean existsByName(String name); 0개의 사용위치 @parkhyeonggyu15

    // TODO: 수정 시 자기 자신(id)을 제외한 상품명 중복 검사용 쿼리메서드를 선언하라.
    // (힌트: boolean existsByNameAndIdNot(String name, Long id))
    boolean existsByNameAndIdNot(String name, Long id); 0개의 사용위치 @parkhyeonggyu15

    // TODO(배치): 품절(stock=0) 상품 건수 조회용 쿼리메서드를 선언하라.
    // (힌트: long countByStock(int stock))
    long countByStock(int stock); 1개 사용 위치 @parkhyeonggyu15
}
```

1. 해당 메서드(scheduleSoldOutCheck) 내부의 DB 작업을 "읽기 전용 트랜잭션"으로 처리하겠다는 선언. 데이터를 변경(insert/update/delete)하지 않고 조회만 할 때 사용.

2.ProductRepository에 등록 시 상품명 중복 검사용 쿼리메서드, 수정 시 자기 자신(id)을 제외한 상품명 중복 검사용 쿼리메서드, 품절(stock=0) 상품 건수 조회용 쿼리메서드 선언.

3.productRepository의 countByStock(0) 메서드를 호출해서 재고(stock)가 0인 상품(즉, 품절 상품)의 개수를 조회. 이 메서드는 보통 long 타입을 반환하는데(JPA 카운트 쿼리는 기본적으로 long), 메서드 반환형이 int이기 때문에 (int)로 캐스팅해서 long → int로 변환한 뒤 반환.

```
@Autowired
private ProductBatch productBatch;

// TODO: 배치 동작 테스트 (NCS 4.3)
// - productBatch.checkSoldOut() 호출
// - 반환된 품절 건수가 0 이상인지 검증(assertTrue(count >= 0))
@Test @kanghyunuk +1 *
void checkSoldOut_동작() {
    int count = productBatch.checkSoldOut();
    assertTrue(condition: count >= 0);
}
```

1. productBatch 객체의 checkSoldOut() 메서드를 호출해서 품절(재고 0) 상품의 개수를 가져와 count 변수에 저장하고, assertTrue(count >= 0)로 count 값이 0 이상인지 검증(assert).

08 | 구현화면

```
2026-06-18T09:12:27.594+09:00 INFO 12432 --- [demo] [ scheduling-1] com.example.demo.Batch.ProductBatch : [Batch] 품질 점검 완료 - 품질 상품 수=0
Hibernate:
    select
        count(p1_0.id)
    from
        tbl_product p1_0
    where
        p1_0.stock=?
2026-06-18T09:13:27.620+09:00 INFO 12432 --- [demo] [ scheduling-1] com.example.demo.Batch.ProductBatch : [Batch] 품질 점검 완료 - 품질 상품 수=0
```

ProductBatch

✓ 테스트 결과

359ms

✓ 1 테스트 통과 총 1개의 테스트, 359ms

```
2026-06-18T09:14:35.072+09:00 INFO 672 --- [demo] [ scheduling-1] com.example.demo.Batch.ProductBatch : [Batch] 품질 점검 완료 - 품질 상품 수=0
WARNING: A Java agent has been loaded dynamically (C:\Users\Administrator\.gradle\caches\modules-2\files-2.1\net.bytebuddy\byte-buddy-agent\1.17.5\58f
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Hibernate:
    select
        count(p1_0.id)
    from
```

ProductBatchTest

08 | 구현화면

테스트 결과419ms

ProductServiceTest419ms

register_정상419ms

테스트가 1개 실패했습니다 총 1개의 테스트, 419ms

tbl_product
(category, create_at, name, price, stock)
values
(?, ?, ?, ?, ?)

register_정상 테스트 성공

ProductServiceTest

- register_정상

테스트 결과758ms

ProductServiceTest758ms

registerBulk_중간실패_전체롤백758ms

테스트가 1개 실패했습니다 총 1개의 테스트, 758ms

tbl_product p1_0

필요:6
실제 :4
<클릭하여 차이점 확인>

> org.opentest4j.AssertionFailedError Create breakpoint: expected: <6> but was: <4> <6 내부 줄>
> at com.example.demo.ProductServiceTest.registerBulk_중간실패_전체롤백(ProductServiceTest.java:64) <1 내부 줄>
> <2개의 접힌 프레임>

ProductServiceTest

- registerBulk_중간실패_전체롤백()
- DB 3개 데이터 존재 할때 기준
 - assertEquals(6,productRepository.count());

08 | 구현화면

The screenshot shows a REST client interface with a GET request to `http://localhost:8090/api/products`. The response is a JSON array of three product objects, each with fields for `id`, `name`, `price`, `stock`, `category`, and `createAt`. The response status is 200 OK, and the body is displayed in JSON format.

```
1  {
2    "number": 0,
3    "totalPages": 1,
4    "content": [
5      {
6        "id": 1,
7        "name": "기계식 키보드",
8        "price": 89000,
9        "stock": 30,
10       "category": "PC주변기기",
11       "createAt": "2026-06-17T12:04:45"
12     },
13     {
14       "id": 2,
15       "name": "무선 마우스",
16       "price": 29000,
17       "stock": 100,
18       "category": "PC주변기기",
19       "createAt": "2026-06-17T12:04:45"
20     },
21     {
22       "id": 3,
23       "name": "27인치 모니터",
24       "price": 245000,
25       "stock": 15,
26       "category": "디스플레이",
27       "createAt": "2026-06-17T12:04:45"
28     }
29   ],
30   "totalElements": 3
31 }
```

ProductRestController 목록조회

ProductRestController 단건조회

The screenshot shows a REST client interface with a GET request to `http://localhost:8090/api/products/2`. The response is a JSON object with the following fields:

```
1  {
2    "id": 2,
3    "name": "무선 마우스",
4    "price": 29000,
5    "stock": 100,
6    "category": "PC주변기기",
7    "createAt": "2026-06-17T12:04:45"
8  }
```

08 | 구현화면

POST

http://localhost:8090/api/products

Send

Params

Authorization

Headers (9)

Body

Pre-request Script

Tests

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

JSON

Beautify

1

2

3

4

5

6

"name": "냉장고",

"price": 2000000,

"stock": 20,

"category": "가전제품"

Body

Cookies

Headers (8)

Test Results

200 OK

118 ms

288 B

Save Response

Pretty

Raw

Preview

Visualize

JSON

1

2

3

"message": "상품 등록 성공!"

Headers			
8 hidden			
	Key	Value	Bulk Edit
☒	Content-Type	application/json	

ProductRestController 등록

08 | 구현화면

POST http://localhost:8090/api/products

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "name": "냉장고",
3   "price": 2000000,
4   "stock": 20,
5   "category": "가전제품"
6 }
```

Body Cookies Headers (8) Test Results 200 OK 118 ms 288 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "message": "상품 등록 성공!"
3 }
```

Headers 8 hidden

	Key	Value	Bulk Edit
☒	Content-Type	application/json	

ProductRestController

등록

- 유효성 검증

```
1 {
2   "name": "삼성휴대폰",
3   "price": -1000000,
4   "stock": -10,
5   "category": "전자제품"
6 }
```

Body Cookies Headers (7) Test Results 400 Bad Request 188 ms 332 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "price": "가격은 0 이상이어야 합니다.",
3   "stock": "재고는 0 이상이어야 합니다."
4 }
```

08 | 구현화면

ProductRestController

수정

• 유효성검증

PUT http://localhost:8090/api/products/4 Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "name": "김치냉장고",
3   "price": 3000000,
4   "stock": 40,
5   "category": "가전제품"
6 }
```

Body Cookies Headers (8) Test Results 200 OK 27 ms 288 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "message": "상품 수정 성공!"
3 }
```

```
1 {
2   "name": "김치냉장고",
3   "price": -3000000,
4   "stock": -40,
5   "category": "가전제품"
6 }
```

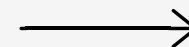
Body Cookies Headers (7) Test Results 400 Bad Request 22 ms 332 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "price": "가격은 0 이상이어야 합니다.",
3   "stock": "재고는 0 이상이어야 합니다."
4 }
```

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 4,
3   "name": "냉장고",
4   "price": 2000000,
5   "stock": 20,
6   "category": "가전제품",
7   "createAt": "2026-06-18T09:30:45"
8 }
```

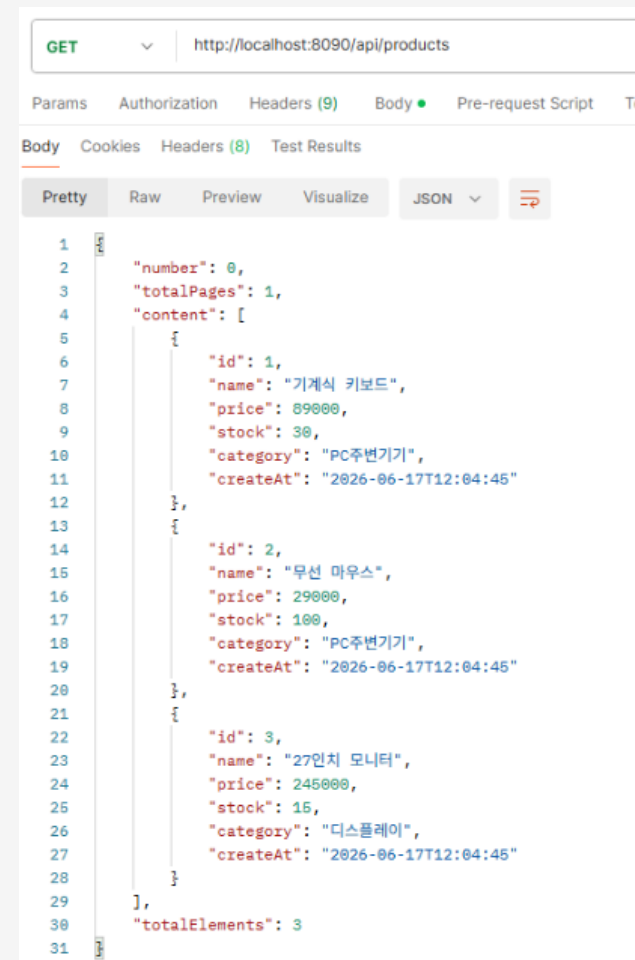
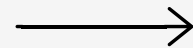
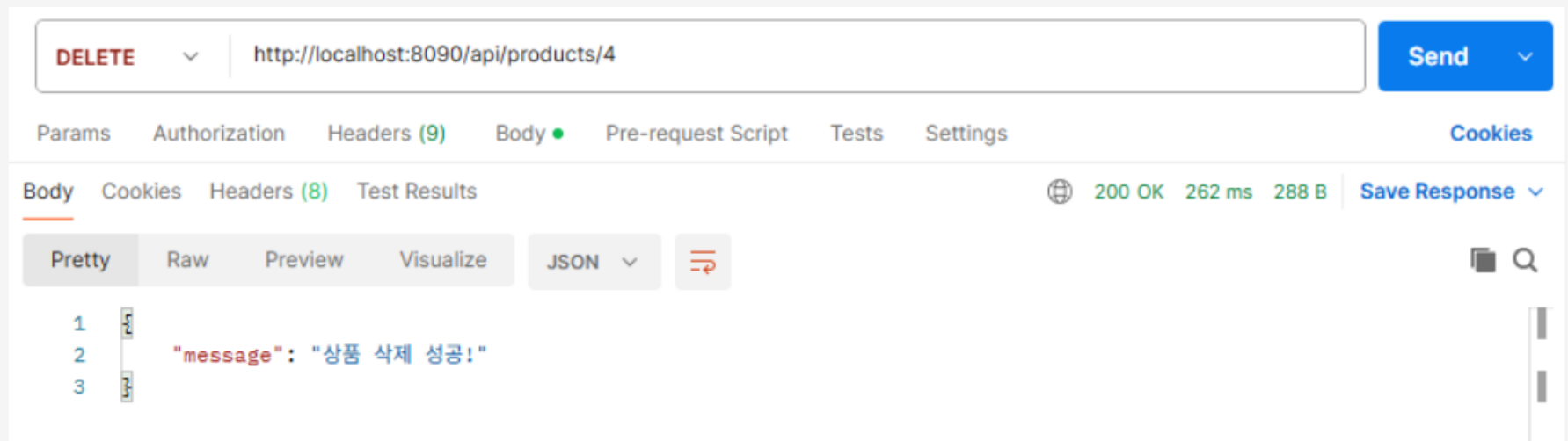


Pretty Raw Preview Visualize JSON

```
1 {
2   "id": 4,
3   "name": "김치냉장고",
4   "price": 3000000,
5   "stock": 40,
6   "category": "가전제품",
7   "createAt": "2026-06-18T09:35:08"
8 }
```

08 | 구현화면

ProductRestController 삭제



08 | 구현화면

POST http://localhost:8090/api/products/bulk

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary JSON Beautify

```
1 [
2   {
3     "name": "냉장고",
4     "price": 2000000,
5     "stock": 20,
6     "category": "가전제품"
7   },
8   {
9     "name": "세탁기",
10    "price": 1500000,
11    "stock": 10,
12    "category": "가전제품"
13  },
14  {
15    "name": "전자레인지",
16    "price": 300000,
17    "stock": 30,
18    "category": "가전제품"
19  }
20 ]
```

Body Cookies Headers (8) Test Results 200 OK 21 ms 298 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "count": 3,
3   "message": "일괄 등록 성공!"
4 }
```

ProductRestController 일괄 등록

GET http://localhost:8090/api/products

Params Authorization Headers (9) Body Pre-request Script Tests Settings

Body Cookies Headers (8) Test Results 200 OK 8 ms 1 KB Save

Pretty Raw Preview Visualize JSON

```
21 {
22   "id": 3,
23   "name": "27인치 모니터",
24   "price": 245000,
25   "stock": 15,
26   "category": "디스플레이",
27   "createdAt": "2026-06-17T12:04:45"
28 },
29 {
30   "id": 4,
31   "name": "냉장고",
32   "price": 2000000,
33   "stock": 20,
34   "category": "가전제품",
35   "createdAt": "2026-06-18T09:40:24"
36 },
37 {
38   "id": 5,
39   "name": "세탁기",
40   "price": 1500000,
41   "stock": 10,
42   "category": "가전제품",
43   "createdAt": "2026-06-18T09:40:24"
44 },
45 {
46   "id": 6,
47   "name": "전자레인지",
48   "price": 300000,
49   "stock": 30,
50   "category": "가전제품",
51   "createdAt": "2026-06-18T09:40:24"
52 }
53 ],
54 "totalElements": 6
55 }
```

08 | 구현화면

POST http://localhost:8090/api/products/bulk

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   {
3     "name": "휴대폰",
4     "price": 2000000,
5     "stock": 20,
6     "category": "전자제품"
7   },
8   {
9     "name": "세탁기",
10    "price": 1500000,
11    "stock": 10,
12    "category": "가전제품"
13  },
14  {
15    "name": "휴대폰",
16    "price": 3000000,
17    "stock": 30,
18    "category": "전자제품"
19  }
20 }
```

Body Cookies Headers (7) Test Results 400 Bad Request 209 ms 287 B Save Response

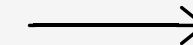
Pretty Raw Preview Visualize JSON

```
1 {
2   "error": "동일한 상품명이 존재합니다."
3 }
```

ProductRestController

일괄 등록

- MyBizException 발생 → 전체 롤백



Result Grid						
Filter Rows:						
	id	name	price	stock	category	create_at
▶	1	기계식 키보드	89000	30	PC주변기기	2026-06-17 12:04:45
	2	무선 마우스	29000	100	PC주변기기	2026-06-17 12:04:45
	3	27인치 모니터	245000	15	디스플레이	2026-06-17 12:04:45
*	NULL	NULL	NULL	NULL	NULL	NULL